

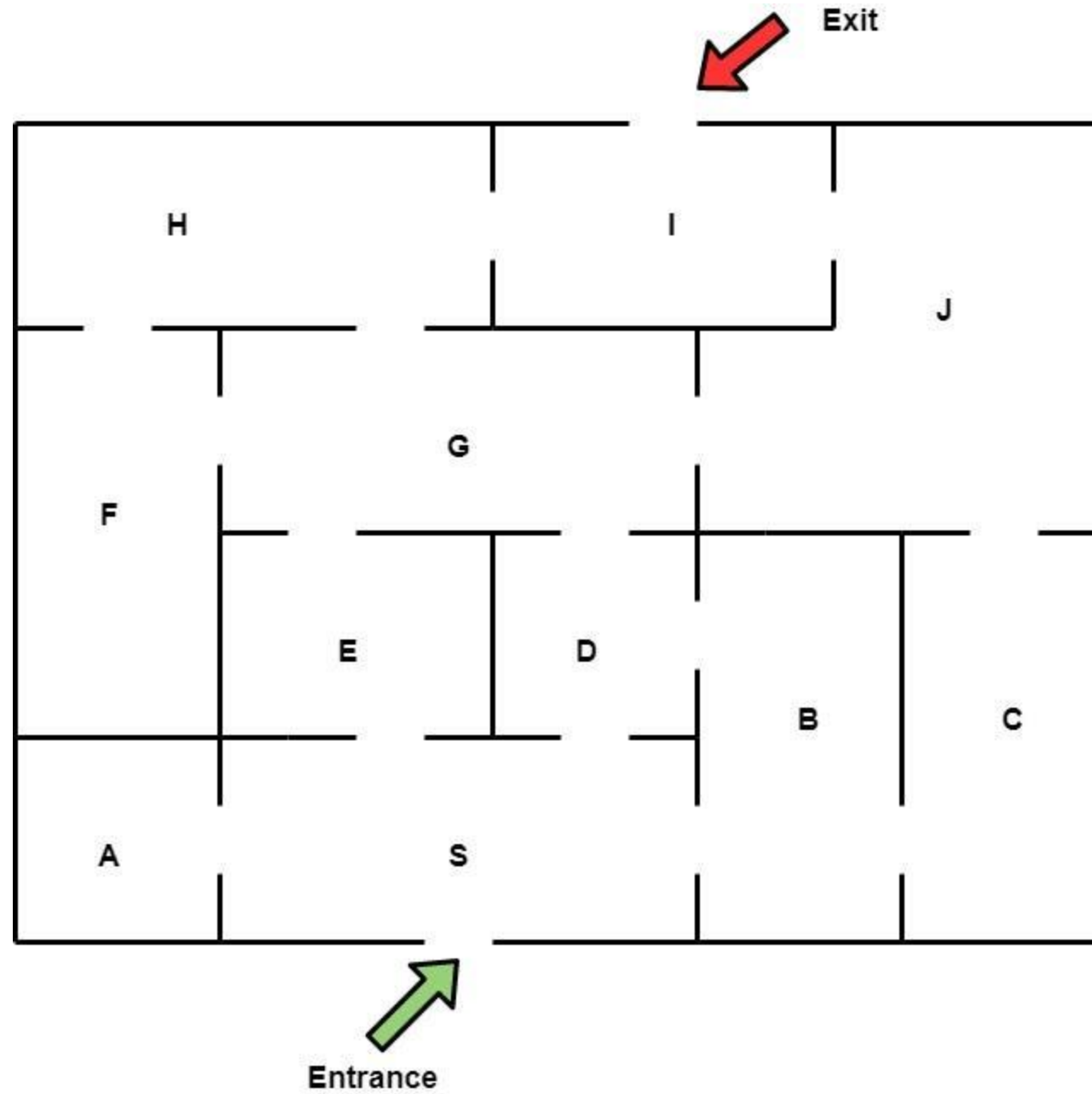
Elementary Graph Algorithms

(BFS)

이원형 교수

Dr. Lee, WonHyong

whlee@handong.edu



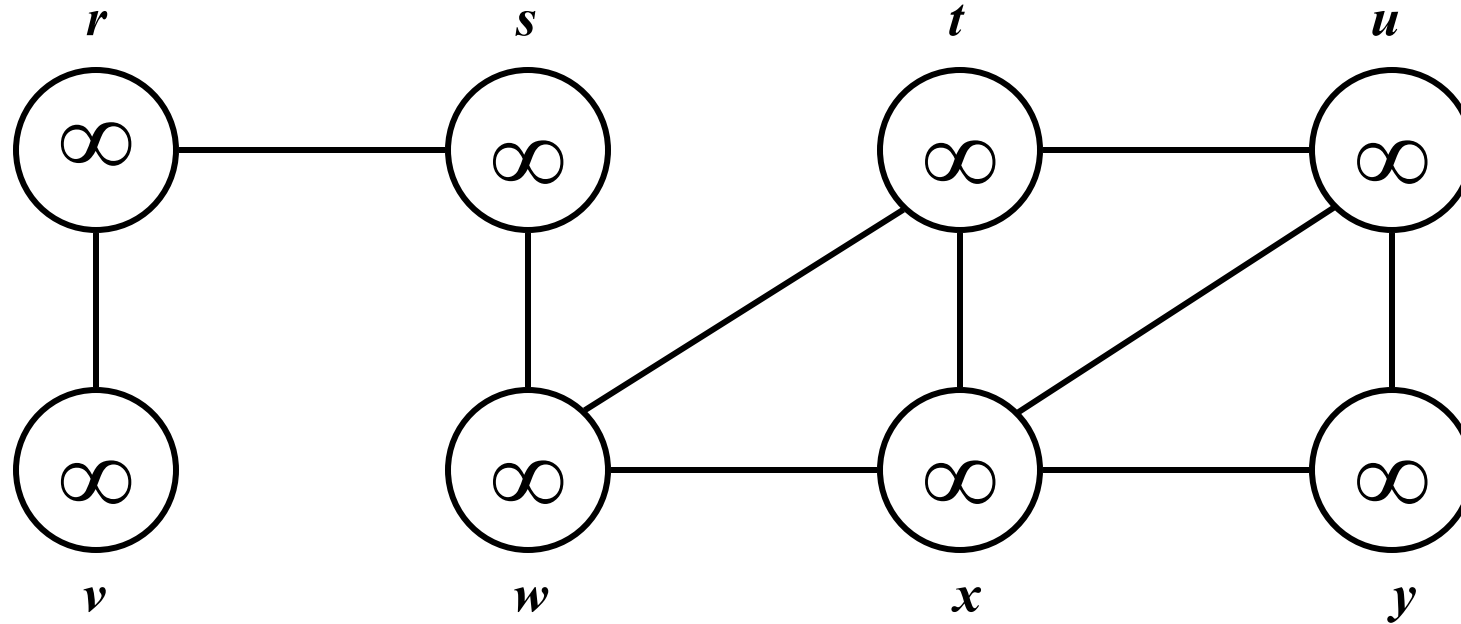
❖ Breadth-First Search (BFS)

- ✓ Scan a graph, and turn it into a 'Breadth-first (spanning) tree'
 - Tree with discovered vertices* and explored* edges.
 - One vertex at a time
 - Pick a source vertex, s to be the root.
 - Find ("discover") its children, then their children, etc.
 - For any vertex reachable from s ,
the path in the tree from s to v corresponds to a "**shortest path**" from s to v in G .
- ✓ * 'Discover' the vertex : The vertex is first encountered.
- ✓ * 'Explore' the edge : The edge is first examined.

BREADTH-FIRST SEARCH

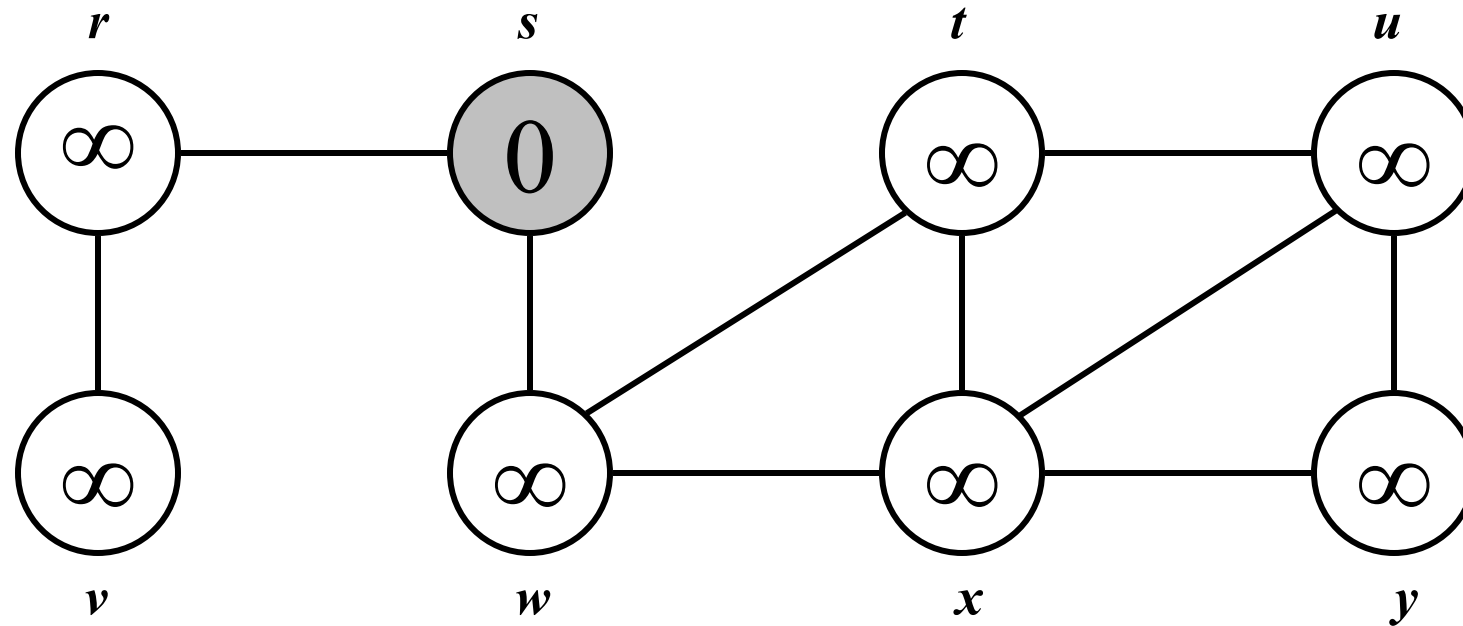
- ❖ Again we will associate vertex “**colors**” to guide the algorithm.
 - ✓ **White** vertices have not been discovered.
 - All vertices start out **white**.
 - ✓ **Gray** vertices are discovered but not finished.
 - They may be adjacent to **white** vertices.
 - The vertex is added to breadth-first tree.
 - ✓ **Black** vertices are discovered and finished.
 - They are adjacent only to **black** and **gray** vertices.
- ❖ As soon as the white vertex is discovered
 - ✓ Turn its color to ‘**gray**’.
 - ✓ Discover all of its **white** neighbor.
 - ✓ After all of its neighbors are discovered, turn its color to **black** (means it is finished).
- ❖ Discover vertices by scanning adjacency list of **gray** vertices.

BREADTH-FIRST SEARCH: EXAMPLE



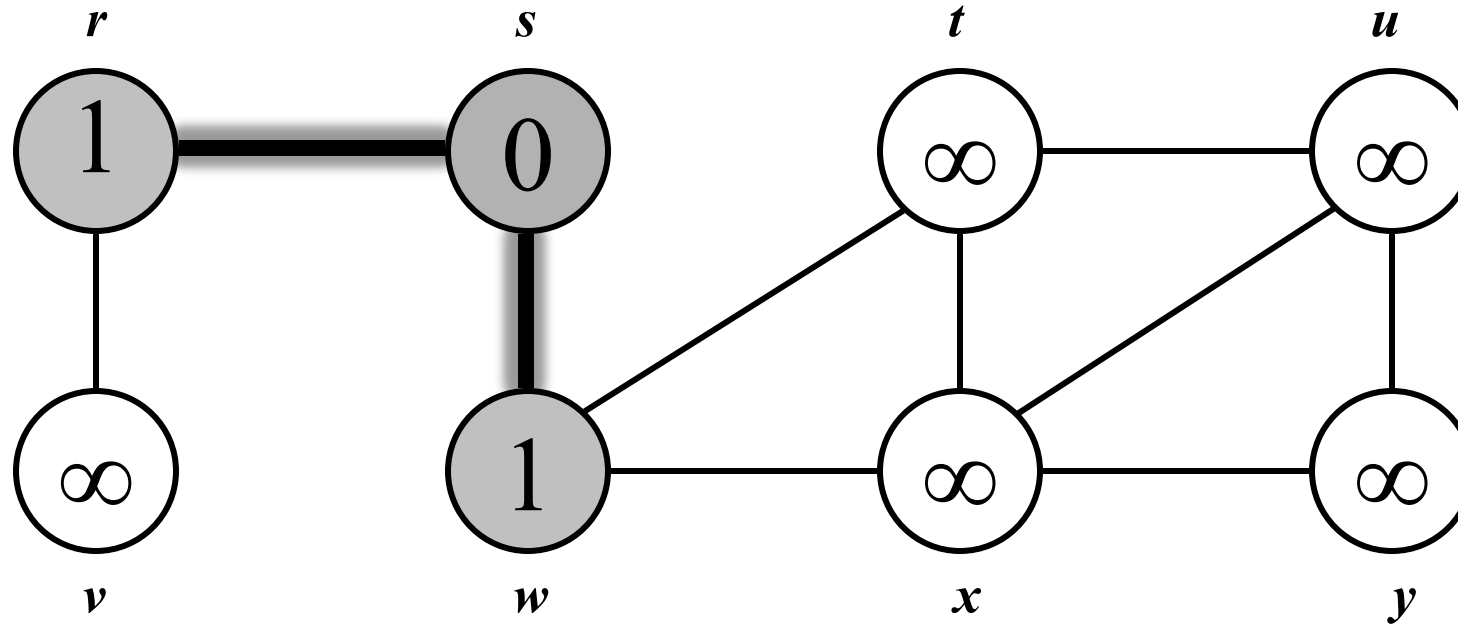
Vertex is an island and edge is a bridge. BFS is visualized as many simultaneous (or nearly simultaneous) explorations from the starting point and spreading out independently.

BREADTH-FIRST SEARCH: EXAMPLE



$Q:$ s

BREADTH-FIRST SEARCH: EXAMPLE

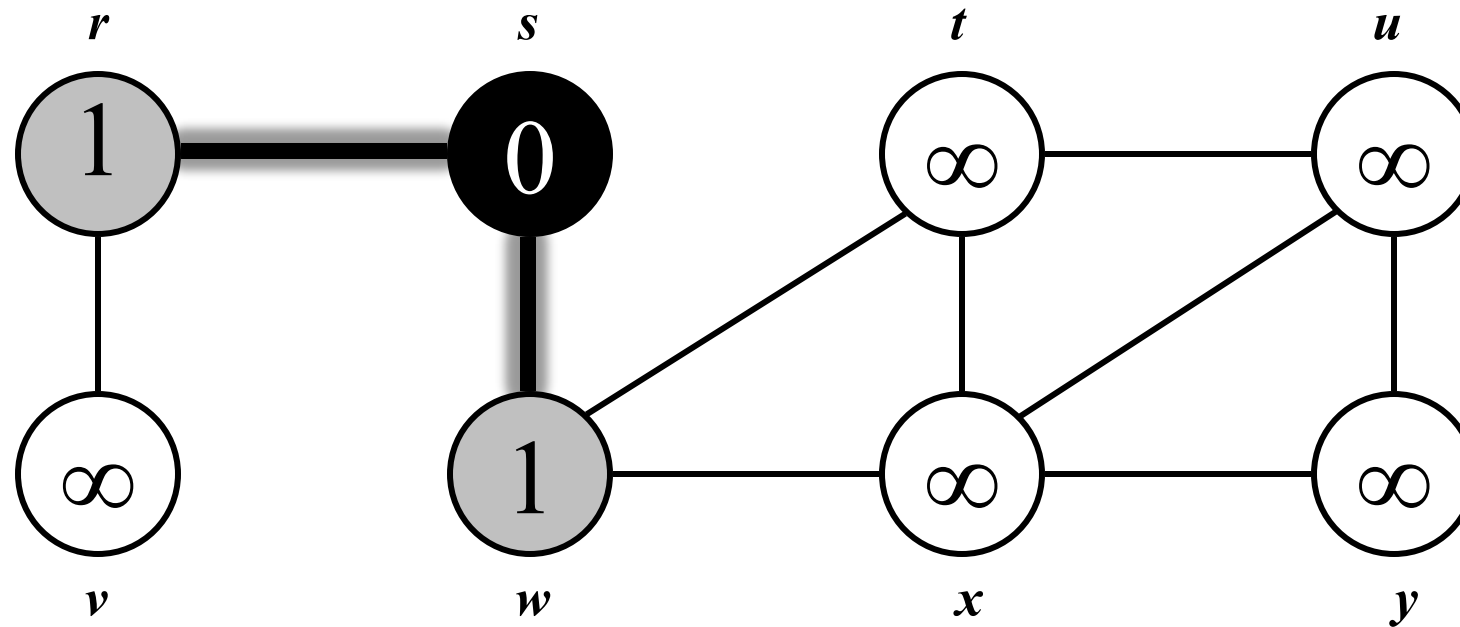


Q :

w	r
-----	-----

- ✓ **White** vertices have not been discovered.
 - All vertices start out **white**.
- ✓ **Gray** vertices are discovered but not finished.
 - They may be adjacent to **white** vertices.
 - The vertex is added to breadth-first tree.
- ✓ **Black** vertices are discovered and finished.
 - They are adjacent only to **black** and **gray** vertices.

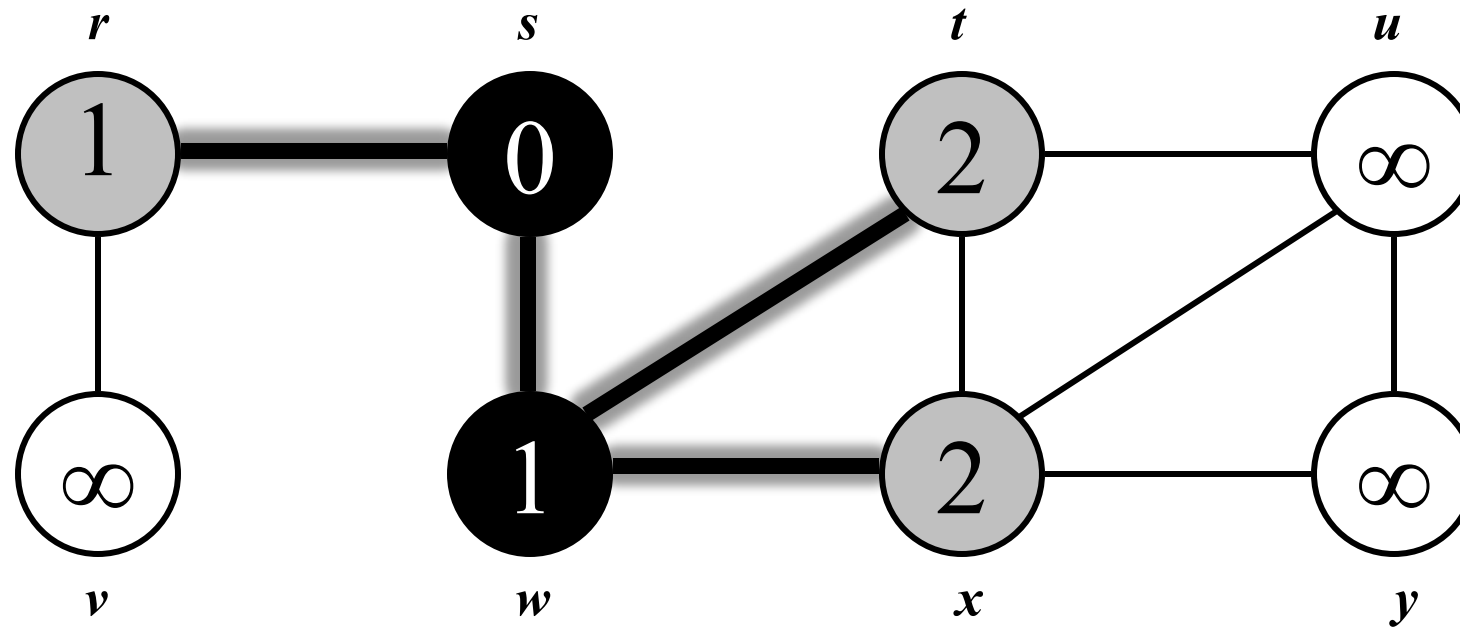
BREADTH-FIRST SEARCH: EXAMPLE



Q:

w	r
----------	----------

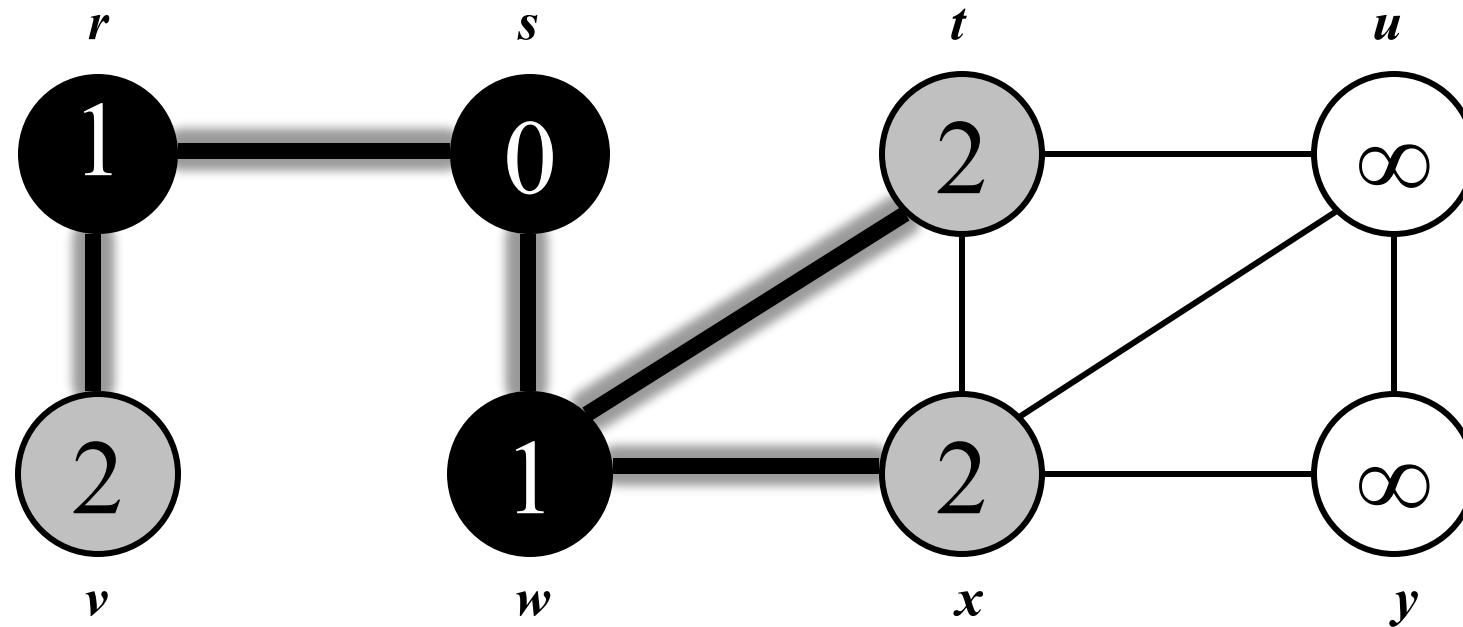
BREADTH-FIRST SEARCH: EXAMPLE



Q :

r	t	x
-----	-----	-----

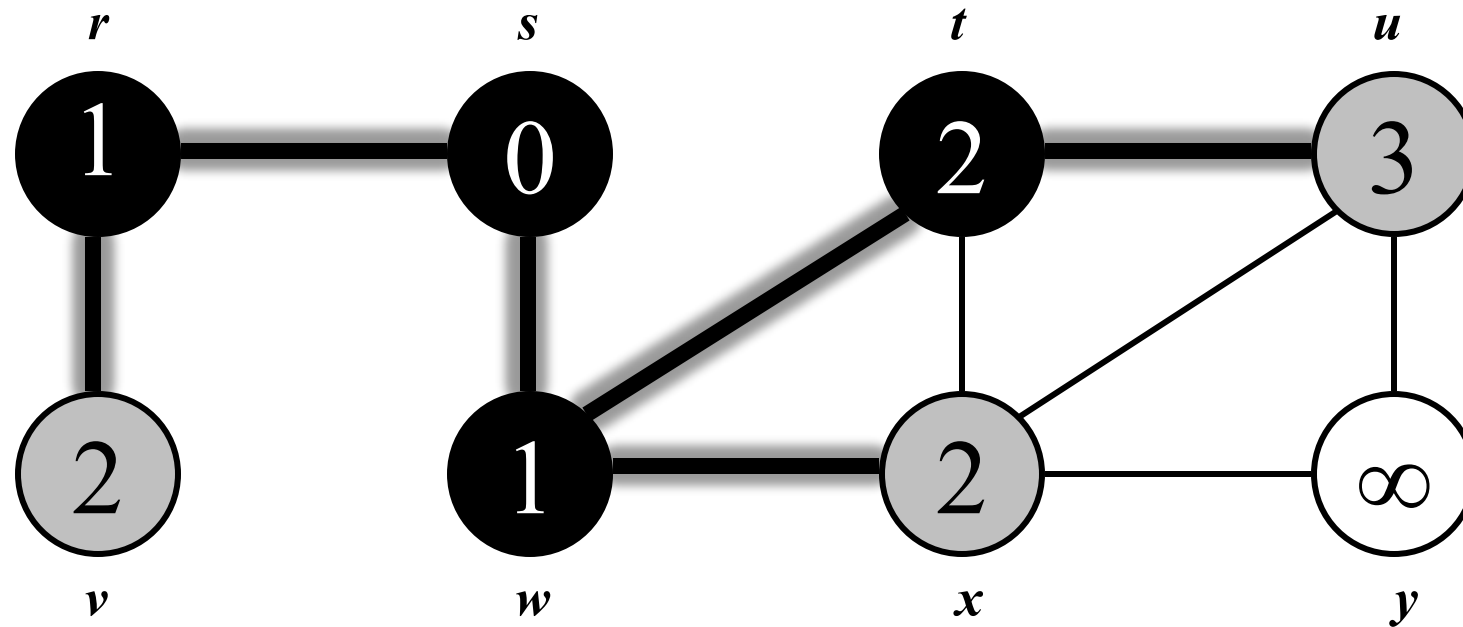
BREADTH-FIRST SEARCH: EXAMPLE



Q :

t	x	v
-----	-----	-----

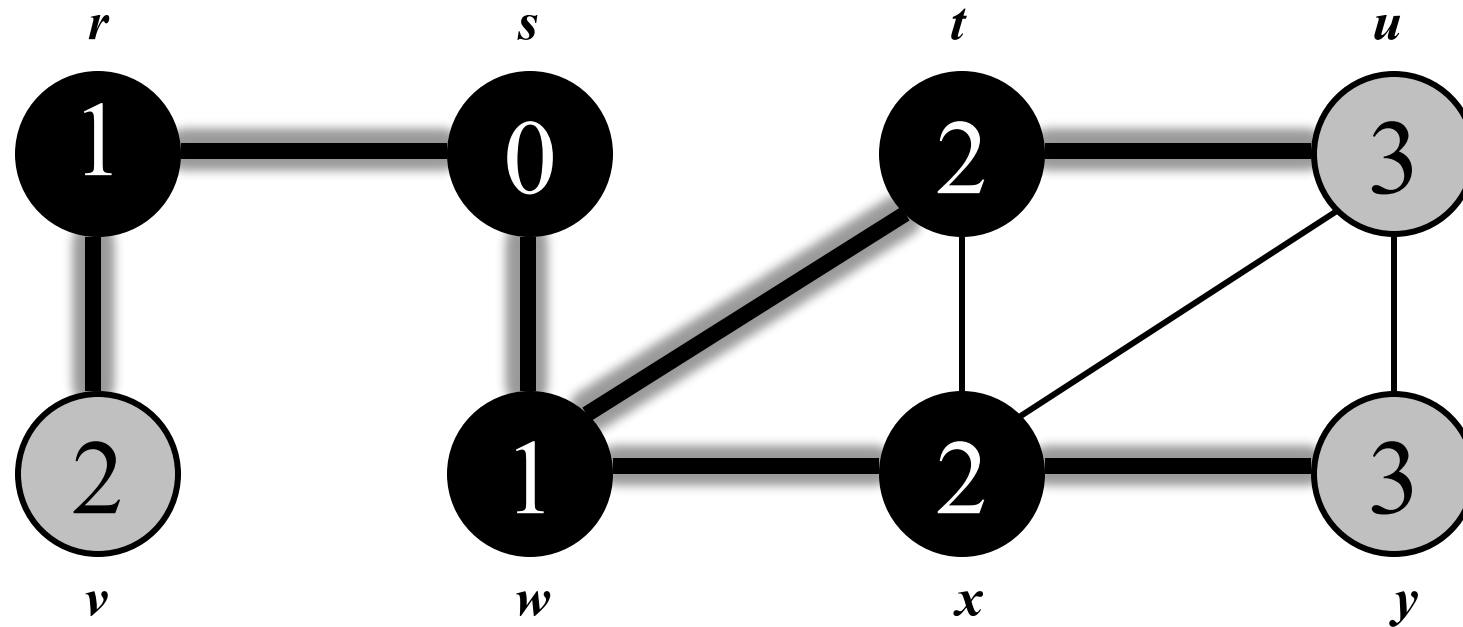
BREADTH-FIRST SEARCH: EXAMPLE



$Q:$

x	v	u
-----	-----	-----

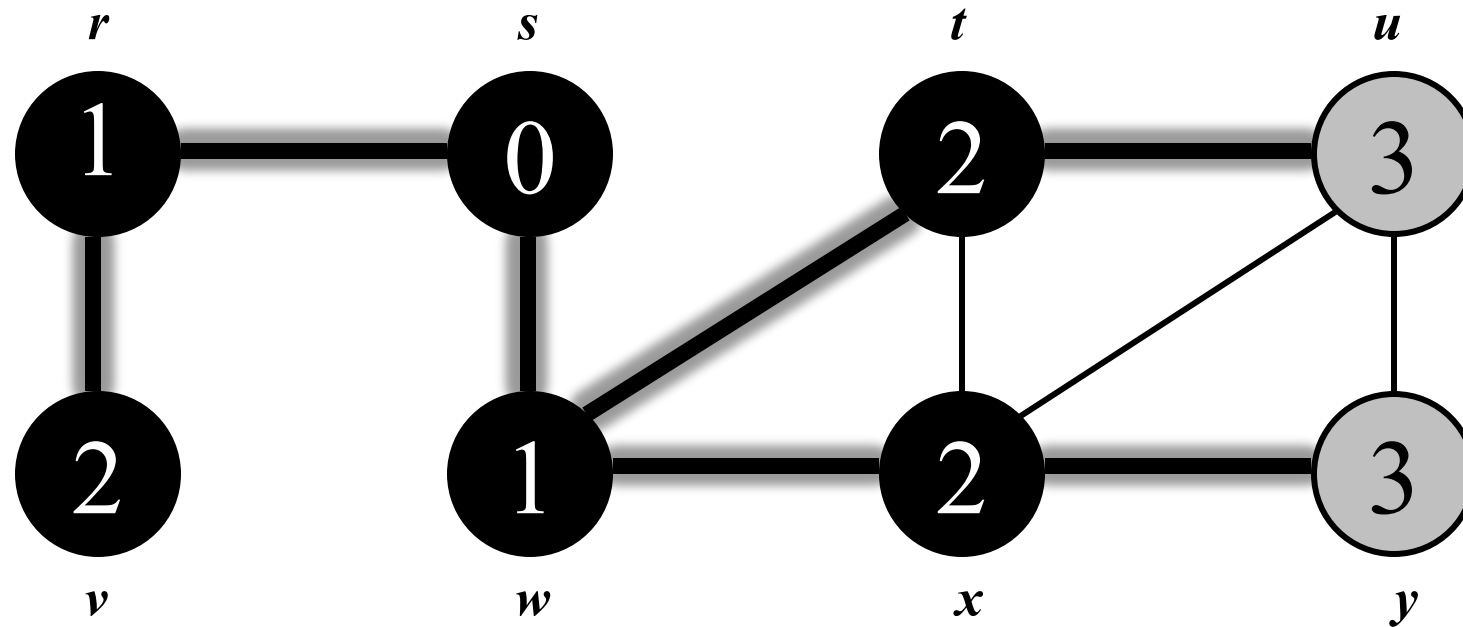
BREADTH-FIRST SEARCH: EXAMPLE



Q :

v	u	y
-----	-----	-----

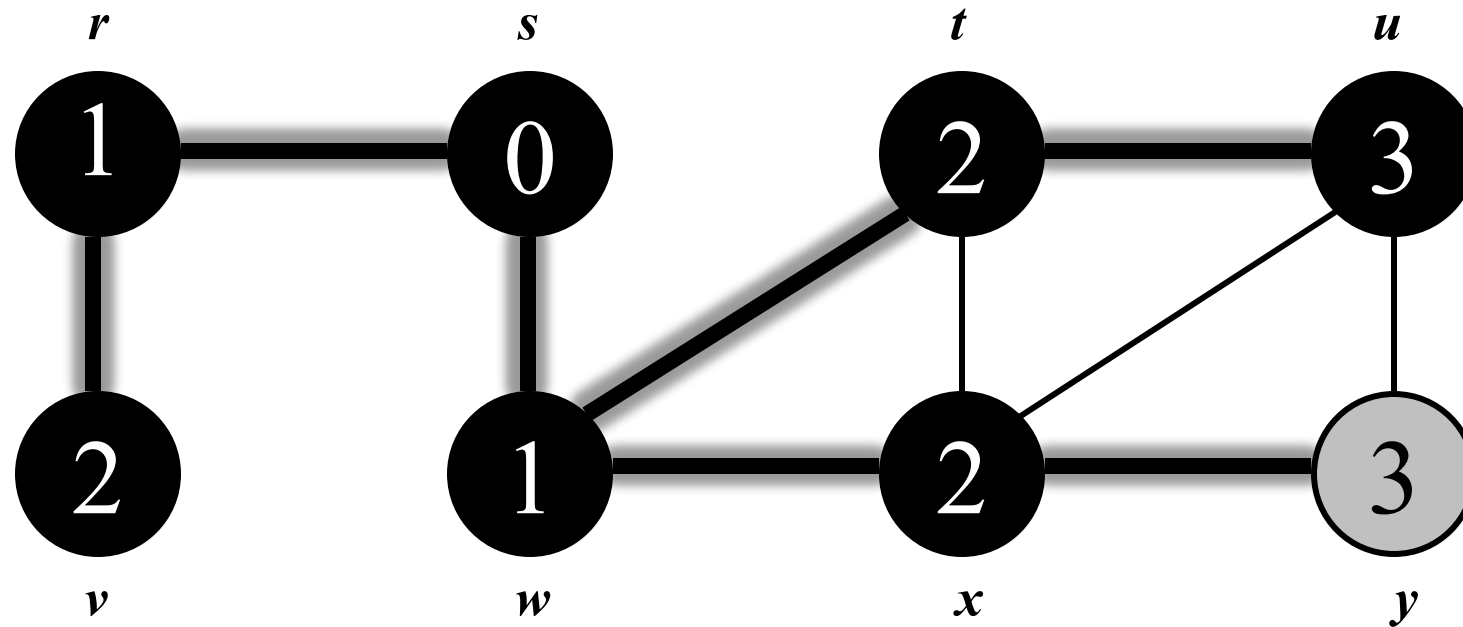
BREADTH-FIRST SEARCH: EXAMPLE



Q :

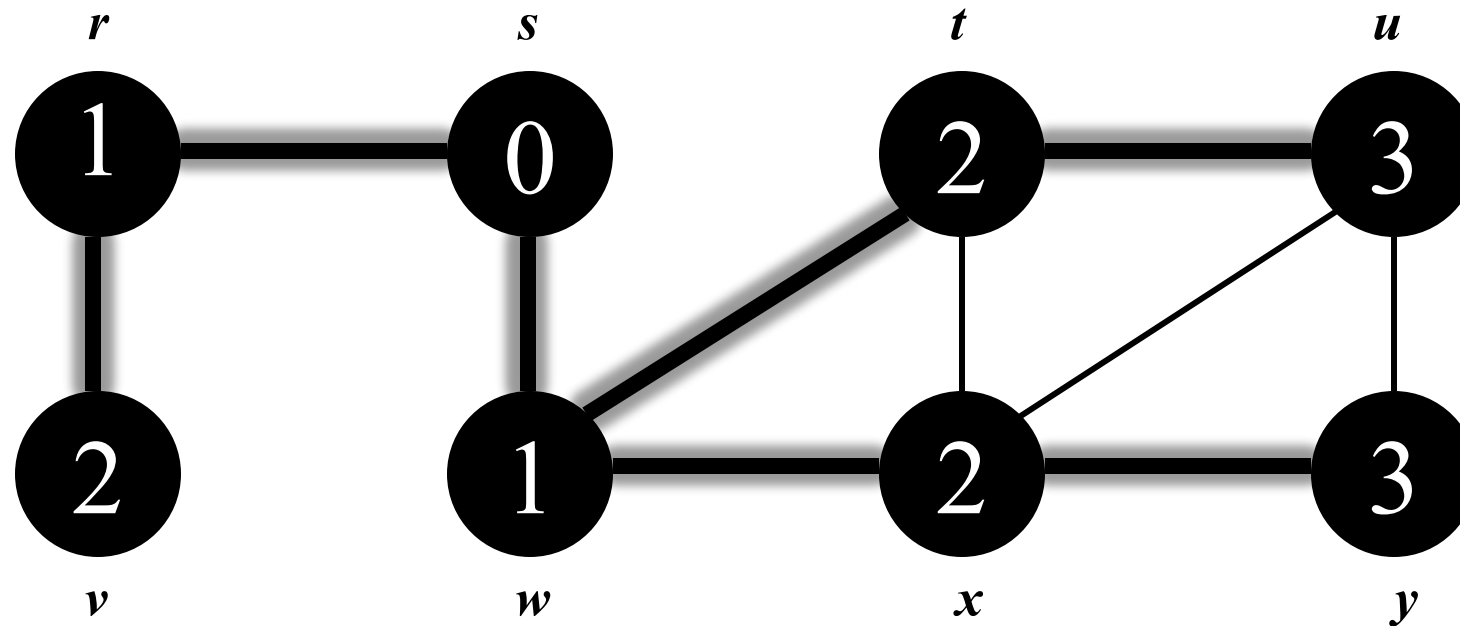
u	y
-----	-----

BREADTH-FIRST SEARCH: EXAMPLE



Q : y

BREADTH-FIRST SEARCH: EXAMPLE



breadth-first tree :

$Q: \emptyset$

consists of vertices and explored edges

BREADTH-FIRST SEARCH

```
BFS( $G, s$ )                                //  $s$ : source vertex
  for each vertex  $u \in V - \{s\}$ 
    do  $d[u] = \infty$ ;  $\pi[u] = \text{NIL}$ ;      //  $\pi$ : predecessor (parent in the tree)
   $color[s] = \text{GRAY}$ ;  $d[s] = 0$ ;  $\pi[s] = \text{NIL}$ ;
   $Q = \emptyset$ 
  ENQUEUE( $Q, s$ )
  while  $Q \neq \emptyset$ 
    do  $u = \text{DEQUEUE}(Q)$ 
      for each  $v \in \text{Adj}[u]$ 
        do if  $color[v] = \text{WHITE}$ 
          then  $color[v] = \text{GRAY}$ 
               $d[v] = d[u] + 1$ 
               $\pi[v] = u$ 
              ENQUEUE( $Q, v$ )
       $color[u] = \text{BLACK}$ 
```


Running time of Breadth-First search:

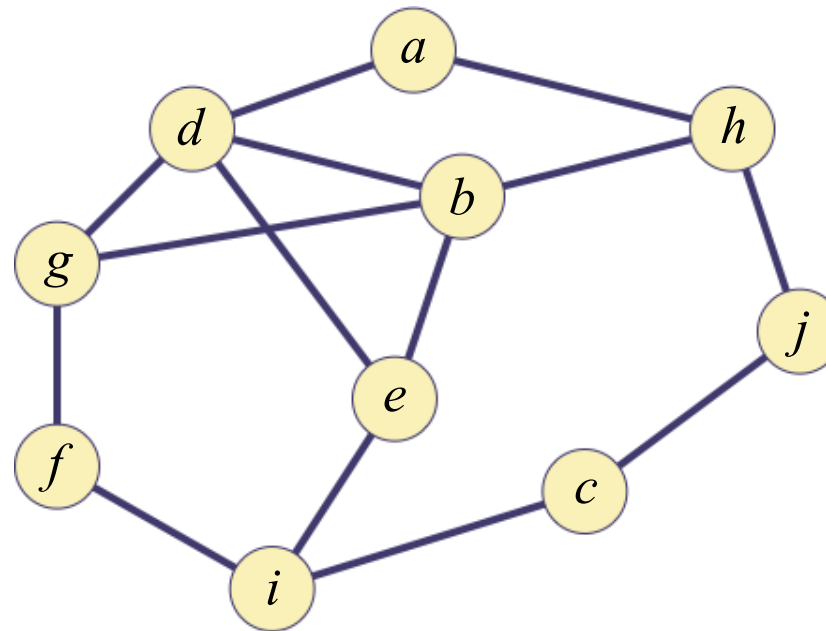
- A. Each node is enqueued once. (white $\rightarrow$ gray) : $\Theta(V)$
- B. Each node is dequeued once. (gray $\rightarrow$ black) : $\Theta(V)$
- C. Each adjacency list is scanned only once. : $\Theta(E)$

Overall running time is $\Theta(V + E)$.

EXERCISE

❖ For the following graph,

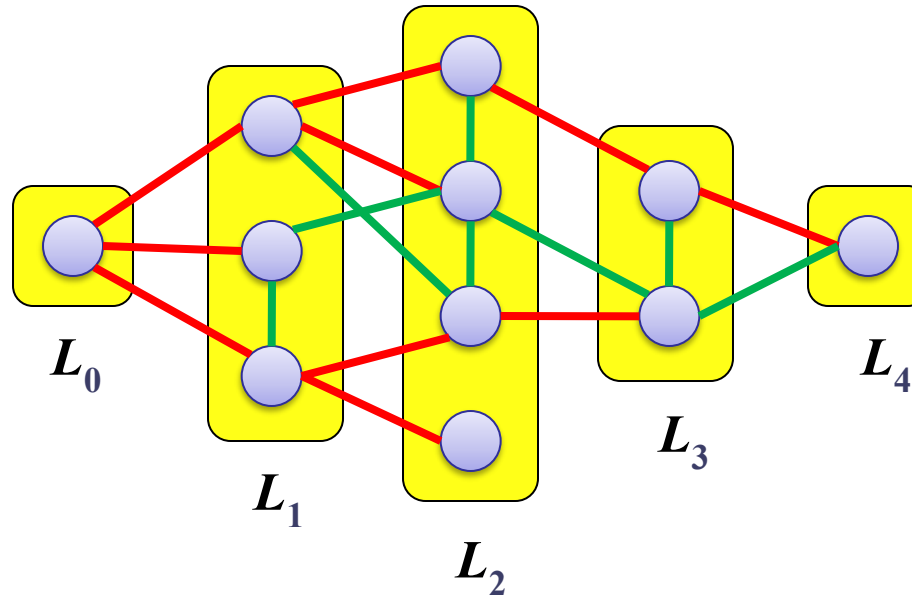
- ✓ Build breadth-first (spanning) tree with vertex ' a ' as root. Assume the vertices are in alphabetical order in the Adj array and that each adjacency list is in alphabetical order.
- ✓ What is the minimum distance of vertex c from the root?



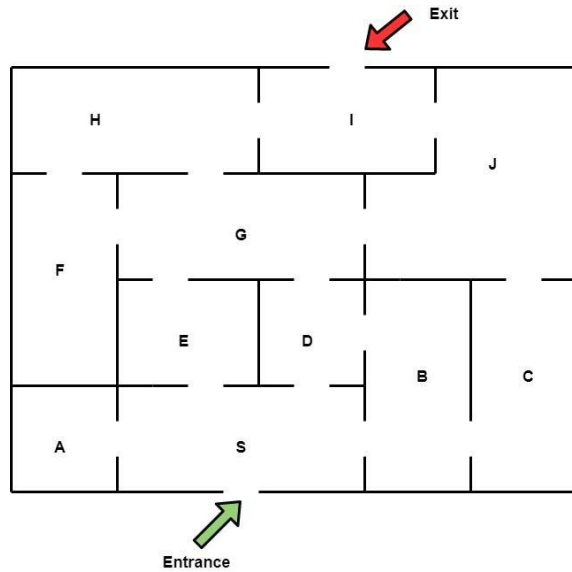
PROPERTIES OF BREADTH-FIRST SEARCH

Theorem: Breadth-first search visits the vertices of G by increasing distance from the root. BFS builds **breadth-first tree**, in which paths to root represent shortest paths in G .

Theorem: *For every edge (v, w) in G such that $v \in L_i$ and $w \in L_j$, $|i - j| \leq 1$.*



MAZE SOLVING USING *BREADTH-FIRST SEARCH*



IGU

Thank you!